

第28章

游戏原型 1：《拾苹果》

是时候了！你将开始制作第一个游戏原型。

因为这是你的第一个游戏原型，所以比较简单。随着深入学习后面的电子游戏原型制作内容，项目会越来越复杂，用到的 Unity 功能也越来越多。

学完本章之后，你将拥有一个可运行的简单的电子游戏。

28.1 电子游戏原型的目的

在着手创建《拾苹果》游戏的原型之前，我们可能需要花点时间考虑一下电子游戏原型的目的。本书第 I 部分花了大量篇幅讨论纸面游戏原型和其为何具有实用性。纸面游戏原型可以帮你完成下面的工作：

- 快速试验、否定或修改游戏机制与规则。
- 探索游戏的动态行为，理解由规则自然产生的可能结果。
- 确保规则和游戏元素易于被玩家理解。
- 了解玩家对游戏的情绪反应。

在电子游戏原型中增加了游戏体验的功能。实际上，这才是它的基本目的。你会花费大量时间向别人详细介绍游戏机制，但让他们试玩这个游戏并亲身体验可能更为有效，以及有趣。Steve Swink 在其书 *Game feel* 中深入讨论了这个问题。¹

在本章中，你将创建一个可玩的游戏，最终成品可以展示给你的好友和同事。在他们玩过之后，你可以问问游戏难度是过于简单、困难，还是刚刚好。使用反馈的信息调整游戏中的参数，为他们定制另一种难度级别。

接下来，让我们开始制作这个《拾苹果》游戏。

为本章创建新项目

按照标准的项目创建流程，在 Unity 中创建一个新项目。如果你需要复习创建项目的标准流程，请参阅附录 A 的“创建项目标准流程”。

1. *Game Feel: A Game Designer's Guide to Virtual Sensation*, 游戏设计经典图书，此书的简体中文版由电子工业出版社出版。



- 项目名称: Apple Picker Prototype
- 场景名称: _Scene_0
- C#脚本名称: ApplePicker、Apple、AppleTree、Basket

不要把 C#脚本绑定到任何对象上。

28.2 准备工作

在第 16 章中,你已经为这个游戏原型做了不少准备工作。在该章中,我们分析了《拾苹果》游戏和经典的《炸弹人》游戏,《拾苹果》的游戏机制与《炸弹人》一样,请花点时间复习一下第 16 章,确认你已经理解了其中的流程图和三个元素:苹果树、苹果和篮子。

建议你在学习本章内容时,每完成一个步骤,就用笔做标注记录。

开始工作:绘图资源

作为一个游戏原型,并不需要漂亮的绘图,它只要能运行即可。在本书中所用到的图片通常称为程序员绘图,它只是由程序员制作出来用于临时充数的图片,最终会被由美工绘制出来的正规游戏绘图取代。与游戏原型中的其他资源一样,这种绘图是为了让你尽快把概念实现为一个可以运行的程序。如果你的程序员绘图看起来不是很糟糕,就算不错了。

苹果树

下面先从苹果树开始。

1. 在菜单栏中执行 GameObject (游戏对象) > 3D Object (3D 对象) > Cylinder (圆柱体) 命令。我们把这个圆柱体当作苹果树的树干。在层级面板中选中国柱体并单击检视面板上方的名称,把它重命名为 Trunk。如图 28-1 所示,设置 Cylinder 的变换组件 (Transform)。

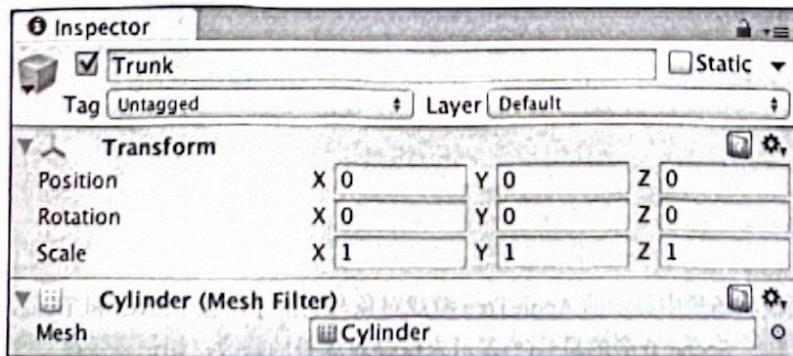


图 28-1 Cylinder 的变换组件

在本书中,使用如下格式教你设置游戏对象的变换组件:



Trunk (Cylinder) P:[0, 0, 0] R:[0, 0, 0] S:[1, 1, 1]

这一行表示这样设置游戏对象 Trunk 的位置：位置 (P) $x=0, y=0, z=0$ ；旋转 (R) $x=0, y=0, z=0$ ；缩放 (S) $x=1, y=1, z=1$ 。括号中的 Cylinder 是游戏对象的类型（圆柱体）。有时候你会在段落中间看到这样的写法：P:[0,0,0] R:[0,0,0] S:[1,1,1]。

2. 在菜单栏中执行 GameObject > 3D Object > Sphere 命令。将 Sphere 重命名为 Leaves，并按下列数字设置其变换组件：

Leaves (Sphere) P:[0,0.5,0] R:[0,0,0] S:[3,2,3]

这个球体和圆柱体结合在一起看起来有点像一棵树，但它们实际上是两个独立的对象。你需要创建一个空白游戏对象作为这两个对象的父对象，这两个对象为子对象。

3. 在菜单栏中执行 GameObject > Create Empty 命令。这样会创建一个空白对象，把它的变换组件设置为：

GameObject (Empty) P:[0,0,0] R:[0,0,0] S:[1,1,1]

空白游戏对象只包含一个变换组件，所以它是容纳其他游戏对象的既简单又实用的容器。

4. 在层级面板中，首先把这个游戏对象重命名为 AppleTree。另一种实现重命名的操作步骤是：单击游戏对象的名称，等待一秒钟，按下回车键（在 Windows 系统中则是按下 F2 键）或者再次单击游戏对象的名称，然后就可以输入新名称了。

5. 分别把 Trunk 和 Leaves 拖动到 AppleTree 上（与把 C#脚本绑定到游戏对象上出现相同的弯箭头的操作方法类似），在层级面板中，它们会出现在 AppleTree 下。你可以单击 AppleTree 旁边的三角形展开按钮查看这两个子对象。完成上述操作之后，你的 AppleTree 游戏对象应该与图 28-2 差不多。

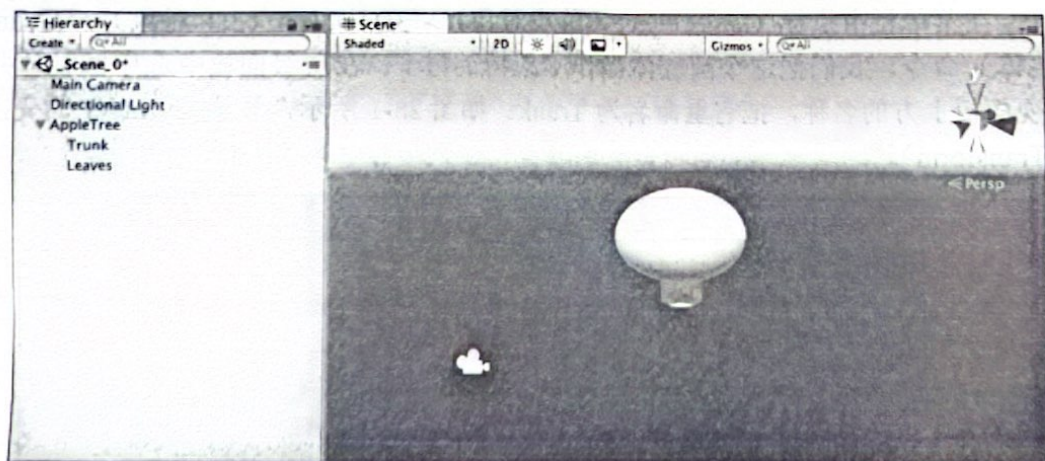


图 28-2 层级面板和场景中显示的 AppleTree 游戏对象与它的子对象 Leaves 和 Trunk，层级面板上方 Scene_0 旁的星号(*)表示未保存对场景的修改，切记要保存

现在 Trunk 和 Leaves 游戏对象都成为了 AppleTree 的子对象，当你移动、缩放或旋转 AppleTree 时，Trunk 和 Leaves 也会随之移动、缩放或旋转。请试着修改 AppleTree



的变换组件,看看不同效果。

6. 在试验结束后,请按以下数值设置 AppleTree 的变换组件。

AppleTree P:[0,0,0] R:[0,0,0] S:[2,2,2]

这个设置会把 AppleTree 移动到中心点,使其尺寸变为最初的 2 倍。

7. 在层级面板中添加 Rigidbody 组件到 AppleTree,并在 Unity 菜单栏中执行 Component > Physics > Rigidbody 命令。

8. 取消勾选 AppleTree 的 Rigidbody 组件的“Use Gravity”选项。如果选中此项,运行场景时苹果树会从天而降。

如第 20 章所述,Rigidbody 组件可以保证在台阶上移动 AppleTree 时,物理仿真环境中的碰撞器 Trunk 和 Sphere 能进行更新。

用于 AppleTree 的简单材质贴图

虽然这只是一个程序员绘图,但它也不能是一个全白物体。下面将为场景添加颜色。

1. 在菜单栏中执行 Assets> Create> Material 命令,在项目面板中创建一个新的材质。

- a. 将这个材质重命名为 Mat_Wood。
- b. 把 Mat_Wood 材质拖动到场景或层级面板中的 Trunk 对象上。
- c. 再次选中 Mat_Wood。
- d. 在检视面板中通过设置“Main Maps”选项,将白色 Mat_Wood 颜色设置为你喜欢的一种棕色。²也可以通过调整 Metallic 和 Smoothness 滑块设置你喜欢的颜色。³

2. 用同样的方法创建一个名为 Mat_Leaves 的材质。

- a. 将 Mat_Leaves 拖动到层级面板或场景中的 Leaves 对象上。
- b. 将白色 Mat_Leaves 设置为绿色。

3. 把 AppleTree 从层级面板拖动到项目面板中,用它创建一个预设。与前面章节中一样,这样会在项目面板中创建一个名为 AppleTree 的预设,并且层级面板中的 AppleTree 会变为蓝色字体。

4. 此时场景中会出现默认的平行光(Directional Light)。按以下数值设置层级面板

2. 使用 Unity Standard Shader 时,阴影的主色调为白色。在 Unity 手册中搜索 Standard Shader,可查看更多详细的信息。从 Unity 文档左侧的目录中就可看出其中几个子章节都有这方面的阐述。搜索“Albedo Color and Transparency”可查看 Albedo 的入门知识。
3. 笔者会设置 Metallic=0、Smoothness=0.25,这样就可以在场景面板中看到 Trunk 的效果,也可以在检视器底部搜索材质预览球体。如果不需要预览,单击检视器底部深灰色的 Mat_Wood 栏。



中光源的位置、旋转和缩放：

Directional Light P:[0,20,0] R:[50,-30,0] S:[1,1,1]

这会在场景中添加一束斜向穿过场景的平行光。注意，这里平行光源的位置并不重要（因为不论平行光源位置在哪，它都会照在所有物体上），但笔者把它设置为 [0,20,0]，是为了把它移出场景中心位置，否则它的图标会出现在场景中心。如果查看光的旋转效果会发现，场景中第一束平行光是和太阳一起绑定在 Unity 默认的 skybox 上的。Apple Picker 不会用到这个设计，但的确是个不错的 3D 游戏效果。

现在，请把 AppleTree 向上移一点，在层级面板中选中 AppleTree 并把它的位置修改为 P:[0,10,0]，这会把它移出场景面板的视野，但你可以滚动鼠标滚轮拉远镜头，这样就可以看到 AppleTree 了。

苹果

现在苹果树已经做好了，你需要为下落的苹果制作游戏对象预设。

1. 在菜单栏中执行 GameObject > 3D Object > Sphere 命令。将球体重命名为 Apple，将变换组件数值按如下设置：

Apple (Sphere) P:[0,0,0] R:[0,0,0] S:[1,1,1]

2. 创建一个名为 Mat_Apple 的材质，设置为红色（如果你喜欢绿苹果，也可设置为浅绿色）。

3. 将 Mat_Apple 拖动到层级面板 Apple 游戏对象上。

为苹果添加物理组件

你也许能回忆起我们在第 17 章中说过的刚体组件可以让对象做出物理反应，例如下落、与其他对象发生碰撞等。

1. 在层级面板中选中 Apple。在菜单栏中执行 Component > Physics > Rigidbody 命令。

2. 单击 Unity 的“播放”按钮，你会看到苹果会在重力作用下落到屏幕之外。

3. 再次单击“播放”按钮停止播放，苹果树会回到初始位置。

为苹果添加 Apple 标签

最后，我们会希望获得屏幕上所有 Apple 游戏对象的一个数组，这可以通过为这些 Apple 游戏对象添加 Apple 标签来实现。

1. 在层级面板中选中 Apple，单击检视面板上的 Tag 旁边的弹出菜单，从中选择 Add Tag（添加标签）菜单项，如图 28-3A 部分所示。这会在检视面板中显示 Unity 的标签和图层管理器（Layers Manager）。

2. 单击 Tags 旁边的三角形展开按钮，查看如图 28-3 B 部分所示的视图显示。

3. 在 New Tag Name 栏（如图 28-3 C 部分所示）中输入 Apple 并保存。此时 Apple



已存在 Tags 列表中（如图 28-3 D 部分所示）。

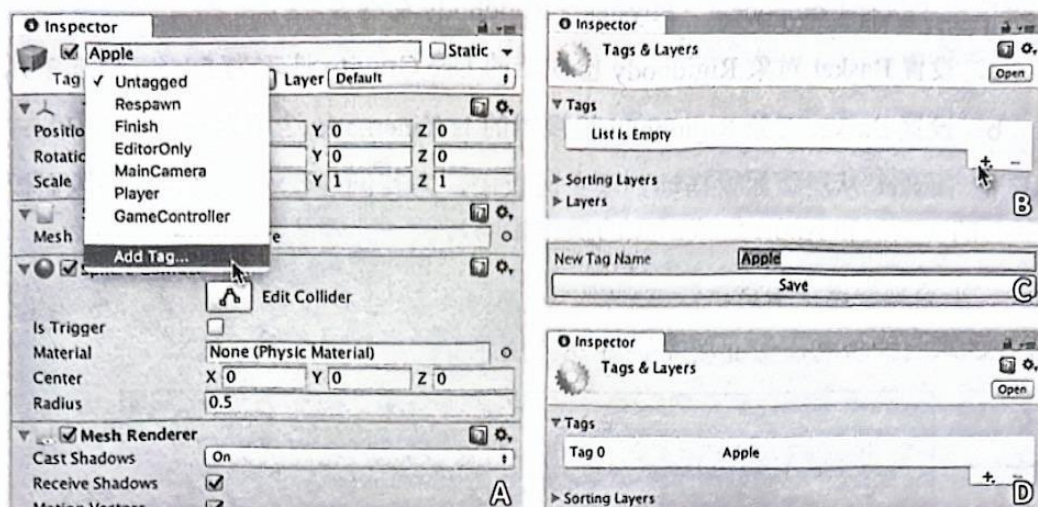


图 28-3 在标签列表中添加 Apple 标签的步骤

4. 现在，在层级面板中单击 Apple，回到 Apple 游戏对象的检视面板。
5. 这时打开 Tag 弹出菜单，可以看到 Tags 列表中多了一个 Apple。从列表中选择 Apple。现在所有的苹果游戏对象都会有一个 Apple 标签，这样就更容易查找和选择它们了。

创建 Apple 预设

按照以下步骤创建 Apple 预设：

1. 把 Apple 从层级面板拖动到项目面板中，为它创建一个预设。⁴
2. 当确认项目面板中存在 Apple 预设之后，就可以从层级面板中选中 Apple 的实例然后删除它了（方法为在右键快捷菜单中选择 Delete 选项，或按下 Command+Delete 组合键，在 Windows 系统中只需按下 Delete 键）。因为在游戏中我们会使用项目面板中的 Apple 预设生成它的实例，所以不必在场景一开始就有一个实例。

篮筐

与其他绘图资源一样，程序员绘制篮筐也非常简单。

1. 在菜单栏中执行 GameObject > 3D Object > Cube 命令，将新创建的立方体重命名为 Basket，并按如下数值设置它的变换组件：

Basket (Cube) P:[0,0,0] R:[0,0,0] S:[4,1,4]

这会生成一个又扁又宽的长方体。

2. 现在新建一个名为 Mat_Basket 的材质，将它的颜色设置为一种低饱和度的浅黄色（类似于稻草颜色），并应用到 Basket 上。

4. 第 19 章详细介绍了有关预设的内容。



3. 将 Rigidbody 组件添加到 Basket 对象中。选中层级面板中的 Basket 对象并在 Unity 菜单栏中执行 Component > Physics > Rigidbody 命令。

a. 设置 Basket 对象 Rigidbody 检视器的 Use Gravity 选项为 false（未选中）。

b. 设置 Basket 对象 Rigidbody 检视器的 Is Kinematic 选项为 true（选中）。

4. 把 Basket 从层级面板拖动到项目面板中，为其创建预设，然后从层级面板中删除 Basket 的实例（和 Apple 游戏对象的操作相同）。

5. 一定要记得保存场景。

现在的项目面板和层级面板应如图 28-4 所示。

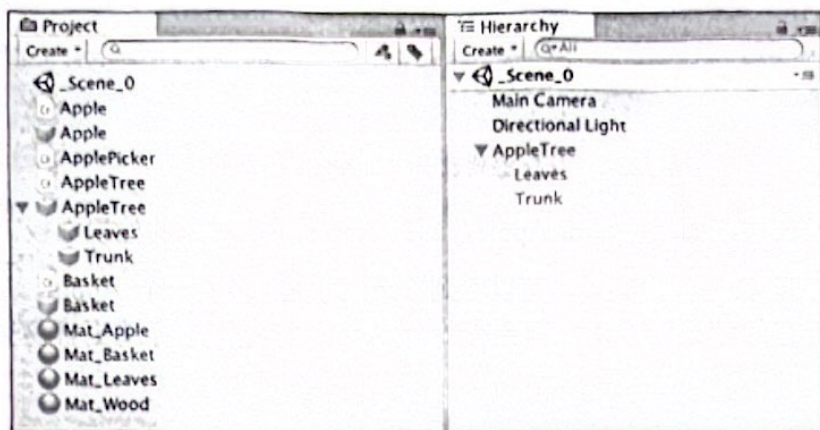


图 28-4 此时项目面板和层级面板的截图，其实在本章开始就应该创建 Apple、ApplePicker、AppleTree 和 Basket 游戏对象脚本作为项目步骤的一部分

摄像机设置

摄像机位置是游戏中最不能出错的内容之一。对于《拾苹果》游戏来说，我们希望摄像机显示一个大小适当的游戏区域。因为这个游戏的玩法完全是二维的，所以我们需要一个正投影（Orthographic）摄像机，而不是透视投影（Perspective）摄像机。

正投影摄像机和透视投影摄像机的对比

如图 28-5 所示，正投影摄像机和透视投影摄像机是游戏中的两类 3D 摄像机。

透视投影摄像机类似人的眼睛，因为光线经过透镜成像，所以靠近摄像机的物体显得较大，而远离摄像机的物体显得较小。这给透视投影摄像机一个平截头四棱锥体（像一个削去尖顶的四棱金字塔）的视野（也称投影）。要查看这种效果，请单击层级面板中的主摄像机，在场景面板中拉远镜头，从摄像机延伸出的金字塔状网格线的视野就是平截头四棱锥体视野，表示摄像机的可视范围。

对于正投影摄像机，物体与摄像机的距离不会影响它的大小。正投影摄像机的投影是一个长方体，而非平截头四棱锥体。要查看这种效果，请在层级面板中选中摄像机，在检视面板中找到 Camera 组件，将 Projection 属性从 Perspective 改为 Orthogonal。现在，灰色的视野范围将是一个三维矩形，而非金字塔形。



有时候将场景面板设置为正投影而非透视投影更有效。做法是：单击场景面板右上角坐标轴手柄下方的 <Persp 字样（如图 28-5 所示）。单击坐标轴手柄下方的 <Persp 字样会在透视和等轴（缩写为 Iso）场景视图间切换（等轴是正投影的同义词）。

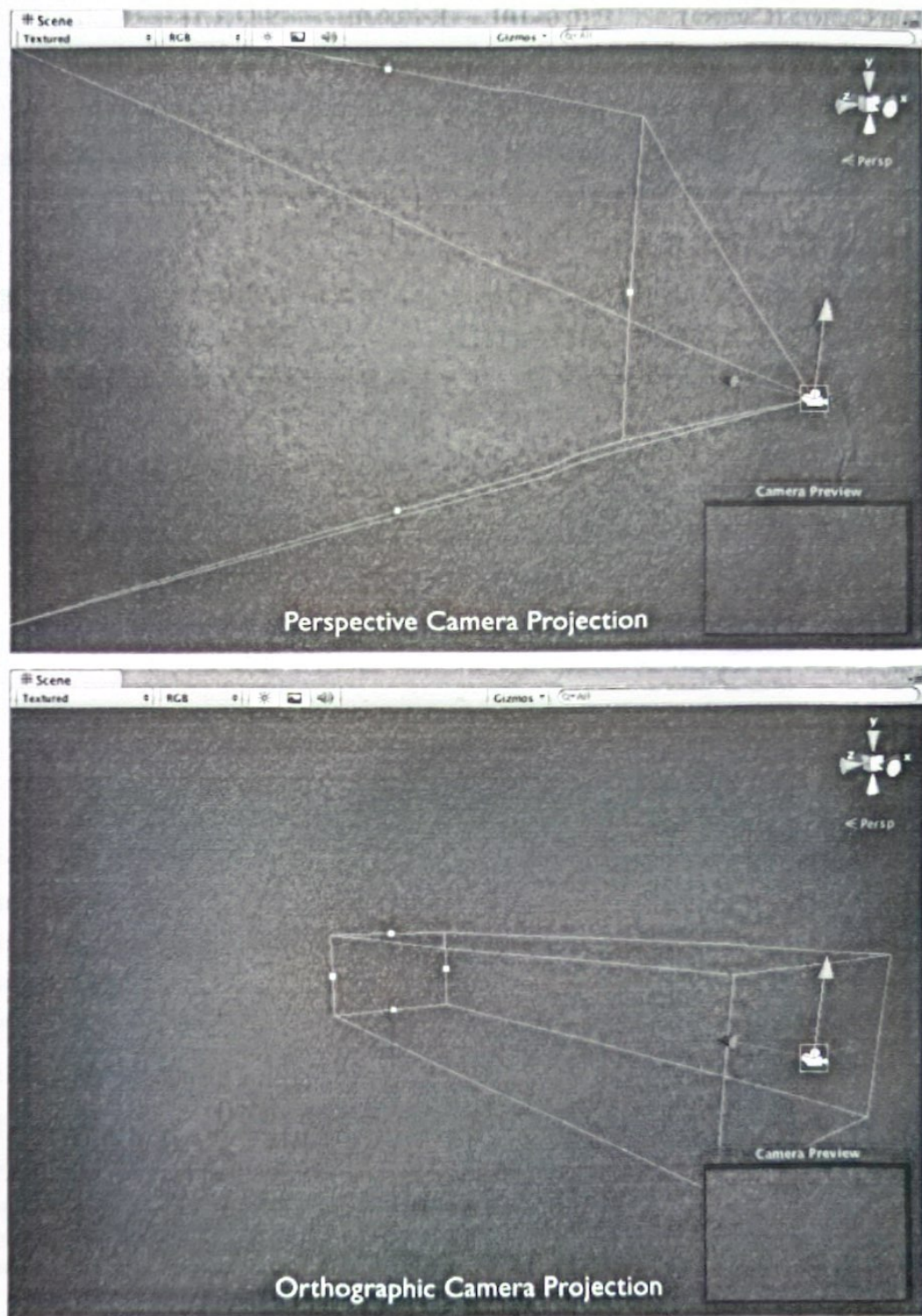


图 28-5 正投影摄像机和透视投影摄像机的对比



《拾苹果》游戏中的摄像机设置

下面将创建《拾苹果》游戏的摄像机设置。

1. 在层级面板中选择主摄像机（Main Camera）并将它的变换组件设置为以下数值：

Main Camera (Camera) P:[0,0,-10] R:[0,0,0] S:[1,1,1]

2. 这会使摄像机的视角下降 1 米（Unity 中的一个单位等于 1 米的长度），正好位于高度为 0 的位置。因为 Unity 的单位等于米，在本书中，笔者有时会使用“单位”代替“米”。

- a. 将 Camera 组件的 Projection 属性改为 Orthographic。

- b. 将 Size 属性设置为 16。

这会使苹果树显示为合适大小，并为玩家抓住下落的苹果留出空间。通常，需要大致猜测一下摄像机设置等数值，然后在试玩游戏的过程中做精细调整。和游戏开发的其他工作一样，要找到最佳的摄像机设置，是一个重复的过程。最终，主摄像机的检视面板如图 28-6 所示。

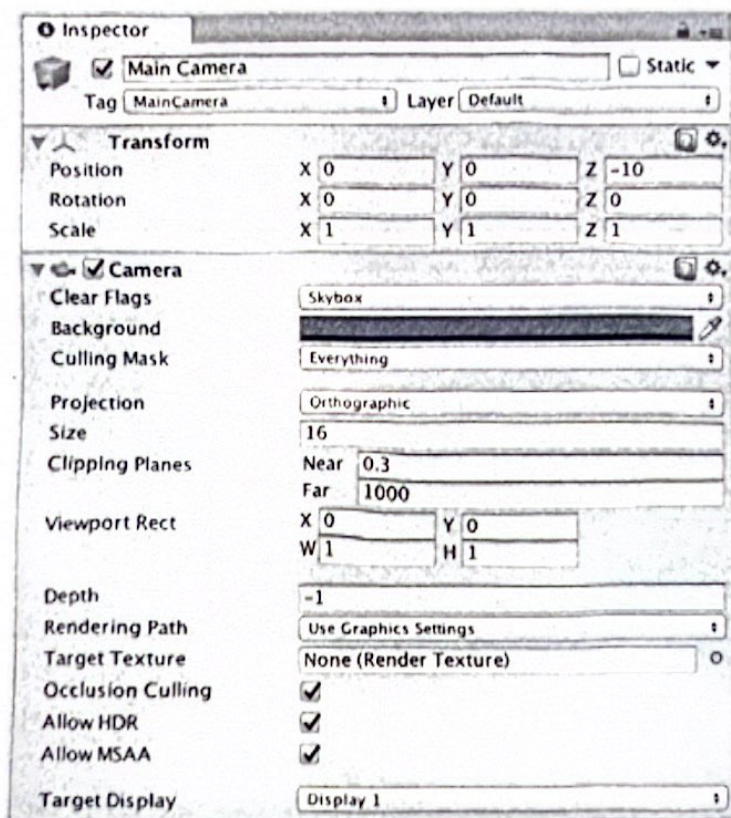


图 28-6 主摄像机的检视面板

游戏面板设置

游戏视图另外一个重要因素是游戏面板的长宽比。

1. 游戏面板顶部的弹出式菜单会显示 Free Aspect，即长宽比设置的菜单。



2. 单击长宽比设置菜单并选择 16 : 9。这是宽屏电视和计算机显示器的标准格式, 全屏玩游戏的效果不错。如果是 macOS 操作系统则应取消勾选低分辨率长宽比 (Low Resolution Aspect Ratios) 选项。

28.3 编写《拾苹果》游戏原型的代码

现在到了编写代码使游戏原型可以运行的阶段。图 28-7 为第 16 章中的苹果树游戏对象的动作流程图。

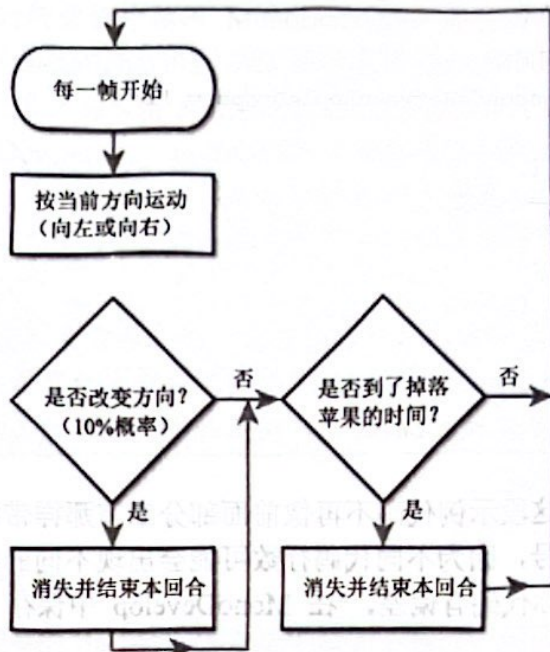


图 28-7 苹果树游戏对象的动作流程图

我们需要为苹果树编写的代码如下：

- 每帧都以一定的速度移动。
- 当碰到游戏区域的边界时改变方向。
- 随机改变方向。
- 每秒落下一个苹果。

就这么简单！接下来，我们开始写代码。在项目面板中双击，打开 AppleTree 脚本。

1. 你需要设置一些变量，所以应该在 MonoDevelop 中选择 AppleTree 类并做如下修改。需添加的代码行以粗体标出。

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class AppleTree : MonoBehaviour {

    [Header("Set in Inspector")]

```




```

// 用来初始化苹果实例的预设
public GameObject applePrefab;

// 苹果树移动的速度
public float speed = 1f;

// 苹果树的活动区域，到达边界时则改变方向
public float leftAndRightEdge = 10f;

// 苹果树改变方向的概率
public float chanceToChangeDirections = 0.1f;

// 苹果出现的时间间隔
public float secondsBetweenAppleDrops = 1f;

void Start () {
    // 每秒掉落一个苹果
}

void Update () {
    // 基本运动
    // 改变方向
}
}

```

你可能会注意到，这段示例代码不再像前面部分章节那样带行号。本书第III部分的示例代码通常不会带行号，因为不同代码行数可能会出现不同的参数返回值，使得笔者的示例代码与读者的实际代码有偏差。⁵在 MonoDevelop 中保存 AppleTree 脚本并返回 Unity。

2. 要真正看到这段代码如何运行，需要把它绑定到 AppleTree 游戏对象上。

- a. 把项目面板中的 AppleTree 脚本拖动到同样位于项目面板的 AppleTree 预设之上。
- b. 单击层级面板中的 AppleTree 实例，此时脚本不但添加到了 AppleTree 预设中，也添加到了层级面板中 AppleTree 预设的实例之中。
- c. 在层级面板中选中 AppleTree 实例，你会看到前面代码中的所有变量都出现在了检视面板中 AppleTree(Script)组件之下。

3. 请尝试修改检视面板变换组件的 X 和 Y 值，在场景中移动 AppleTree 的位置，找到苹果树的最佳垂直位置（position.y）和左右移动的边界。在笔者的计算机上，position.y 的最佳位置是 12，苹果树的水平位置在 -20 到 20 之间时仍能显示在屏幕上。

- a. 设置 AppleTree 的位置为 [0,12,0]。

5. 省略行号的另一个原因是笔者需要在每行显示出更多字符。



- b. 设置检视面板 AppleTree(Script)组件的 leftAndRightEdge 变量值为 20。

Unity 引擎脚本参考

在深入学习本项目之前,如果你对上述代码有任何问题,最好查看一下 Unity 脚本参考。查看脚本参考有两种方法:

1. 在菜单栏中执行 Help > Scripting Reference (脚本参考) 命令。这会打开浏览器并显示存放在计算机上的脚本参考文档,即使计算机没有联网也能查看。你可以在搜索框中输入任何函数名或类名查看详细信息。

在脚本参考网页的搜索框中输入 MonoBehaviour 并按回车键,然后单击第一个搜索结果,查看每个 MonoBehaviour 脚本的所有内置方法(这些方法,也是你编写并绑定到 Unity 游戏对象上的所有 MonoBehaviour 派生类的内置方法)。

2. 在使用 MonoDevelop 时,选择想深入了解其含义的任何文本,然后在菜单栏中执行 Help > Unity API Reference (Unity API 参考) 命令,这样会弹出一个联机的 Unity 脚本参考,只能联网查看,但它的内容与第一种方法看到的本地参考是一模一样的。

你在第一次访问脚本参考页面时,需要从窗口右上方的长方形按钮区选择 C# 或 JavaScript 语言。确保在窗口右上方选择的是 C# 语言。Unity 很多示例代码既可以是 C# 也可以是 JavaScript,而且很多老的示例只有 JavaScript 版本。

基本运动

下面对运动做如下修改:

1. 在 AppleTree 脚本的 Update() 方法中做如下修改,修改内容以粗体字标出。注意代码行中的省略号。这是笔者为了缩减书籍篇幅而省略掉的代码行。你在使用时千万不要删掉相应代码!

```
public class AppleTree : MonoBehaviour {
    ... //a
    void Update () {
        // 基本运动
        Vector3 pos = transform.position; //b
        pos.x += speed * Time.deltaTime; //c
        transform.position = pos; //d

        // 改变方向
    }
}
```

上述代码的解释如下。

- a. 笔者在示例代码中使用省略号代表隐藏的代码行,不然后面章节的代码将会很长。你在看到这类省略号时,请保持代码原封不动,仅关注增加的代码(粗体显示)就行。这里的省略号表示不需要修改 AppleTree 类声明语句和



Update() 方法之间隐藏的代码行。

- b. 定义了一个三维向量变量 pos，使其等于 AppleTree 的当前位置。
- c. 本行让 pos 的 x 组件增加 speed 和 Time.deltaTime 的乘积（后者是从上一帧到现在的秒数）。这样使 AppleTree 基于时间运动，这在游戏编程中是一个重要概念（见“让游戏中的运动基于时间”专栏）。
- d. 将这个修改后的 pos 赋值给 transform.position（使 AppleTree 移动到新的位置）。如果不将 transform.position 设置为 pos，苹果树是不会移动的。

让游戏中的运动基于时间

让游戏中的运动基于时间，是指不管游戏的帧速率为多少，运动都保持恒定的速度。通过 Time.deltaTime 可以实现这一点，因为它能告诉我们从上一帧到现在经历了多少时间。Time.deltaTime 通常非常小。对于 25fps（帧/秒）的游戏来说，Time.deltaTime 为 0.04f，即每帧的时间为 4/100 秒。如果 //b 行代码以 25fps 的帧速率运行，结果将解析为：

```
pos.x += speed * Time.deltaTime;
pos.x += 1.0f * 0.04f;
pos.x += 0.04f;
```

因此，在 1/25 秒的时间内，pos.x 将递增 0.04 米。在 1 秒钟内，pos.x 将增加 0.04 米/帧 × 25 帧，即 1 米/秒。

如果游戏以 100fps 的帧速率运行，该行代码将解析为：

```
pos.x += speed * Time.deltaTime;
pos.x += 1.0f * 0.01f;
pos.x += 0.01f;
```

所以，在 1/100 秒时间里，pos.x 每帧将递增 0.01f 米。在 1 秒钟的时间里，pos.x 的增量为 0.01 米/秒 × 100 帧，合计 1 米/秒。

不管游戏的帧速率为多少，基于时间的运动都可以保证游戏元素以恒定速度运动，这样可以保证游戏在最新配置和老配置的计算机上都可以玩。基于时间的编程在开发移动游戏时非常重要，因为移动设备的配置变化得非常快。

你可能会困惑为什么要使用三行代码完成这一功能，而不是一行。为什么代码不能这样写：

```
transform.position.x += speed * Time.deltaTime;
```

答案是 transform.position 是一个属性，即通过 get() 和 set() 存取器伪装成字段（例如将函数功能伪装为变量）的方法（参见第 26 章）。尽管可以读取属性子部件的值，但是不能直接设置属性的子部件。换句话说，transform.position.x 可读，但不可直接写。所以必须创建一个过渡的三维向量 pos，对这个变量做出修改，然后赋值给 transform.position。



2. 保存脚本, 返回 Unity 并单击“播放”按钮。你会看到 AppleTree 开始慢慢移动。你可以在检视面板中为 speed 设置几个不同的值, 看看哪个速度感觉更舒服。笔者将它设置为 10, 使它以 10 米/秒的速度运动 (每秒 10 米或者每秒 10 个 Unity 单位)。停止播放并在检视器中设置 speed 为 10。

改变方向

现在 AppleTree 能够以适当的速度运动了, 但它很快就会“跑”出屏幕。我们需要让它在达到 leftAndReightEdge 值时改变方向。请将 AppleTree 脚本做如下修改:

```
public class AppleTree : MonoBehaviour {
    ...
    void Update () {
        // 基本运动

        ...

        // 改变方向
        if ( pos.x < -leftAndRightEdge ) {           //a
            speed = Mathf.Abs(speed); // 向右运动      //b
        } else if ( pos.x > leftAndRightEdge ) {      //c
            speed = -Mathf.Abs(speed); // 向左运动     //c
        }
    }
}
```

- 本行代码检查上一行中新设置的 pos.x 是否小于 leftAndRightEdge 设置的边界。
- 如果 pos.x 太小, 则将 speed 设置为 Mathf.Abs(speed), 即 speed 的绝对值, 确保它是一个正数, 速度为正, 表示向右运动。
- 如果 pos.x 大于 leftAndRightEdge, 则将 speed 设置为 Mathf.Abs(speed), 确保 AppleTree 向左运动。

保存脚本, 返回 Unity, 单击“播放”按钮查看效果。

随机改变方向

按如下步骤加入随机改变方向的功能:

- 添加下面的粗体代码:

```
public class AppleTree : MonoBehaviour {
    ...
    void Update () {
        // 基本运动

        ...

        // 改变方向
        if ( pos.x < -leftAndRightEdge ) {
            speed = Mathf.Abs(speed); // 向右移动
```




```

    } else if ( pos.x > leftAndRightEdge ) {
        speed = -Mathf.Abs(speed); // 向左移动
    } else if ( Random.value < chanceToChangeDirections ) { //a
        speed *= -1; //改变方向 //b
    }
}
}

```

a. Random.value 返回一个 0 到 1 之间的浮点数（包括 0 和 1，即结果有可能是 0 或者 1）。

b. 如果 a 中的随机数小于 ChanceToChangeDirections，苹果树的运动速度会变为它的相反数。

2. 单击“播放”按钮，你会看到当 ChanceToChangeDirections 为 0.1f 时，方向改变得太过频繁。在检视面板中，将 ChanceToChangeDirections 改为 0.02，感觉会好很多。

继续根据“让游戏中的运动基于时间”专栏内容探讨基于时间的游戏，这里运动发生改变的概率其实并非是基于时间的。在每一帧中，苹果树都有 2% 的概率改变方向。在性能强的计算机上，有可能每秒运行 400 帧（导致平均每秒改变 8 次方向），而在性能较弱的计算机上，每秒可能只运行 30 帧（导致平均每秒改变 0.6 次方向）。

3. 要解决这一问题，需要把改变方向的代码从 Update() 函数中移到 FixedUpdate() 函数中，前者的调用频率与计算机渲染画面帧的速度一致，而后者不论计算机性能强弱，每秒均运行 50 次。

```

public class AppleTree : MonoBehaviour {
    ...
    void Update () {
        // 基本运动

        ...

        // 改变方向
        if ( pos.x < -leftAndRightEdge ) {
            speed = Mathf.Abs(speed); // 向右运动
        } else if ( pos.x > leftAndRightEdge ) {
            speed = -Mathf.Abs(speed); // 向左运动
        } //a
    }
    void FixedUpdate() {
        // 随机改变运动方向
        if ( Random.value < chanceToChangeDirections ) { //b
            speed *= -1; // 改变方向
        }
    }
}

```

a. 剪切步骤 1 中标记为 a 和 b 的两行代码，以右花括号代替。



b. 粘贴到此处。

这将导致 AppleTree 平均每秒随机改变 1 次方向 (每秒 50 次 FixedUpdate $\times$ 0.02 的随机概率=每秒 1 次)。

掉落苹果

下面将介绍如何实现苹果的掉落。

1. 在层级面板中选中 AppleTree, 然后在检视面板中查看 Apple Tree(Script)组件。现在, applePrefab 字段值为 None (Game Object), 即暂未设置 (括号里的 GameObject 是让我们知道 applePrefab 字段的类型是游戏对象)。这个值应设置为项目面板中的 Apple 预设。可以通过下面两种操作方法实现:

- 单击 Apple Prefab None (Game Object)右侧的小圆圈, 然后从资源选项卡中选择 Apple。
- 或者把项目面板中的 Apple 预设拖动到检视面板中的 Apple Prefab 中。第 19 章的图 19-4 详细描述了相关步骤。

2. 返回 MonoDevelop 窗口, 将下面以粗体字表示的代码添加到 AppleTree 类中:

```
public class AppleTree : MonoBehaviour {
    ...
    void Start () {
        // 每秒掉落一个苹果
        Invoke( "DropApple", 2f ); //a
    }

    void DropApple() { //b
        GameObject apple = Instantiate<GameObject>( applePrefab ); //c
        apple.transform.position = transform.position; //d
        Invoke( "DropApple", secondsBetweenAppleDrops ); //e
    }

    void Update () { ... } //f
    ...
}
```

- a. Invoke() 函数以固定间隔调用某个函数。本行调用新的 DropApple() 函数。第二个参数 2f 通知 Invoke() 在调用 DropApple() 函数之前先等待 2 秒。
- b. DropApple() 为自定义函数, 可在苹果树所在位置实例化 Apple 对象。
- c. DropApple() 创建 applePrefab 对象并将值赋给 apple 游戏对象。
- d. 新的 apple 游戏对象位置将设置为苹果树的位置。
- e. 再次调用 Invoke()。此时会以 secondsBetweenAppleDrops 秒的间隔调用 DropApple() 函数 (本行的 secondsBetweenAppleDrops 值为检视器默认设置的 1 秒)。因为 DropApple() 的自身调用机制, 因此游戏运



行的效果为每秒都会掉落苹果。

- f. 本行的{ ... }表示笔者隐藏了 Update() 方法的代码。当你看到省略号时，不需要修改任何对应的代码。

3. 保存苹果树脚本，返回 Unity，然后单击“播放”按钮查看效果。

你能预见到苹果会往旁边掉落吗？与第 19 章的示例中所有的立方体都四处乱飞是同样的问题。笔者举这个例子是为了教会你如何修复此类问题。首先将苹果树的 Rigidbody 组件设置为 kinematic，即可以通过代码使苹果移动而不会与其他物体碰撞。

4. 在苹果树检视器的 Rigidbody 组件中，勾选 Is Kinematic。

5. 再次单击“播放”按钮，此时苹果仍然会发生碰撞问题。

虽然修复了苹果树碰撞问题，但苹果与苹果树仍然会发生碰撞，使苹果向两边掉落，而非垂直下落。要解决这个问题，你需要把苹果放在与苹果树不会发生碰撞的物理图层（Physics Layer）中。物理图层是指对象的分组，我们可以规定各组对象之间是否会发生碰撞。如果苹果树和苹果放在两个不同的物理图层中，并在图层管理器中规定两个图层不发生碰撞，这样苹果和苹果树也不会再撞到一起了。

设置游戏对象物理图层

首先，你需要创建几个新物理图层，步骤如下。

1. 在层级面板中单击 AppleTree，然后在检视面板中 Layer 旁边的下拉菜单中选择 Add Layer（添加图层）命令。这将在检视面板中打开标签和图层管理器（Tags and Layers Manager），可以从中设置 Layers 标签下物理图层的名字（要看清楚，不要编辑 Tags 或者 Sorting Layers 标签下的内容）。可以看到，从 Layer 0 到 Layer 7 是内置图层，显示为不可编辑的灰色。但可以编辑 Layer 8 到 Layer 31 之间的图层。

2. 请将 Layer 8 命名为 AppleTree，Layer 9 命名为 Apple，Layer 10 命名为 Basket。



图 28-8 创建物理图层（步骤 1 和 2）以及为对象分配图层（步骤 5）的操作步骤

3. 在菜单栏中执行 Edit > Project Settings > Physics 命令，在检视面板中显示物理管理器（Physics Manager）。物理管理器下方由复选框构成的图层碰撞矩阵表用来设置哪



些物理图层可以互相碰撞，以及同一物理图层中的对象是否可以互相碰撞。

4. 我们希望苹果与苹果树或其他苹果都不发生碰撞，但仍然需要与篮筐碰撞，所以图层碰撞矩阵表设置应该如图 28-9 所示。

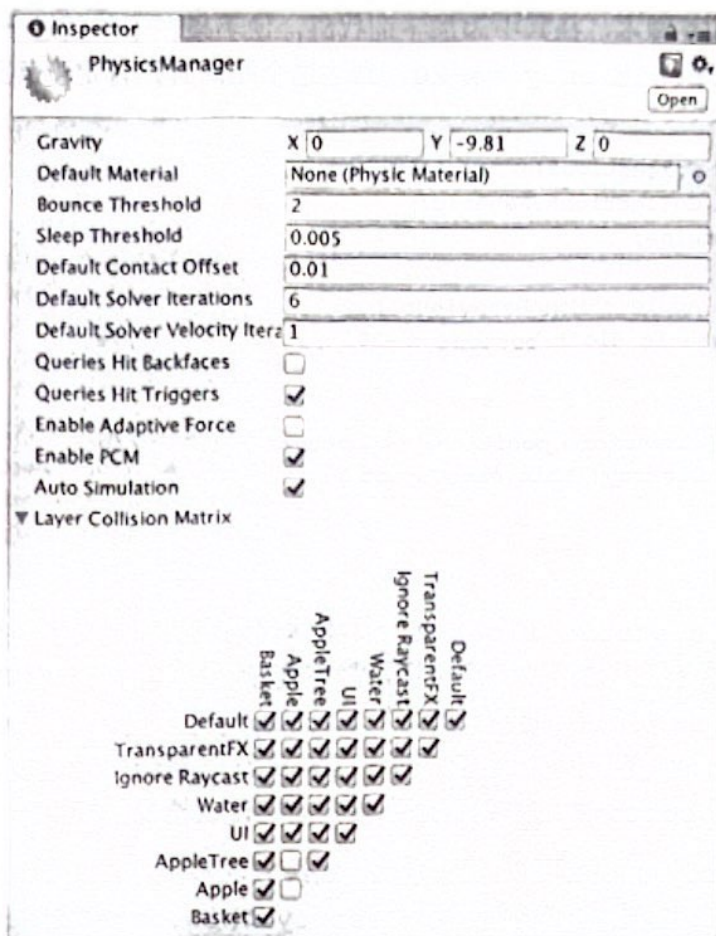


图 28-9 在物理管理器中，应按图中所示设置图层碰撞矩阵表

5. 正确设置完图层碰撞矩阵表之后，需要将游戏中的重要对象分配到图层之中。

- 在项目面板中单击 Apple 预设。然后在检视面板上部从 Layer 旁边的下拉菜单中选择 Apple 图层。
- 在项目面板中选中 Basket，将其图层设置为 Basket。
- 然后在项目面板中选中 AppleTree，将其图层设置为 AppleTree。

当选择 AppleTree 的物理图层时，Unity 会提示是否同时修改 AppleTree 和其子对象的图层，这里肯定要选 Yes，因为需要修改构成 AppleTree 树干和树冠的圆柱和圆球所在的物理图层。这一修改也会传递给场景中的 AppleTree 实例，可以单击层级面板中的 AppleTree 实例加以确认。

现在单击“播放”按钮，你会看到苹果以正常方式从树上落下。



在苹果下落一定距离后销毁苹果

如果让目前的程序多运行一会儿，你会发现在层级面板中出现了很多 Apple 对象。这是因为程序每秒都会创建一个新的 Apple 实例，但从来不销毁它们。

1. 打开 Apple 脚本，在其中添加下面代码，使苹果在下落超过一定距离之后被销毁（transform.position.y == -20，到达这个距离时，肯定下落到屏幕之外了）。以下为实现代码：

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class Apple : MonoBehaviour {
    public static float bottomY = -20f;           //a

    void Update () {
        if ( transform.position.y < bottomY ) {    //b
            Destroy( this.gameObject );
        }
    }
}
```

- a. **public static float** 这一行粗体代码声明和定义了一个名为 bottomY 的静态变量。如第 26 章所讲，静态变量会被类的所有实例所共享，因此所有 Apple 实例中都会有相同的 bottomY 变量值。如果修改一个实例中的 bottomY 变量值，所有实例中的该变量值都会同时改变。但需要指出的是，像 bottomY 这样的静态字段不会出现在检视面板中。
- b. Destroy() 函数可以销毁游戏传递给它的参数，可用于销毁组件和游戏对象。在这里，必须使用 Destroy(this.gameObject)，因为使用 Destroy (this) 只能销毁 Apple 游戏对象实例的 Apple(Script)组件。在任何脚本中，this 都代表调用该语句的 C#类的当前实例（在本例中为 Apple 类），而非整个游戏对象。如果要从绑定的组件类代码中销毁整个游戏对象，必须调用 Destroy(this.gameObject)。

2. 保存 Apple 脚本。

3. 要运行这个 Apple 脚本，需要将它绑定到项目面板中的 Apple 预设之上。现在你已经学会把脚本拖动到预设上方这种方法，下面介绍另一种方法：

- a. 在项目面板中选择 Apple。
- b. 滚动到检视器底部，单击“Add Component（添加组件）”按钮。
- c. 在下拉菜单中执行 Scripts > Apple 命令。

现在单击 Unity 的“播放”按钮，你会看到苹果会下落一段距离，一旦 Y 值小于 -20，苹果就会消失。

至此，就是有关苹果的所有内容。



实例化篮筐

要使篮筐对象工作,我们需要介绍一个概念,这个概念会在本书教程中反复出现。尽管面向对象的思维鼓励我们为每个游戏对象单独创建一个类(如 AppleTree 和 Apple),但有时也需要在一个脚本中控制整个游戏的运行。

1. 把 ApplePicker 绑定到层级面板中的主摄像机上。笔者经常把这类游戏管理脚本绑定到主摄像机上,因为笔者确信每个场景都有一个主摄像机。

2. 在 MonoDevelop 中打开 ApplePicker 脚本,输入以下代码并保存:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class ApplePicker : MonoBehaviour {

    [Header("Set in Inspector")] // a
    public GameObject basketPrefab;
    public int numBaskets = 3;
    public float basketBottomY = -14f;
    public float basketSpacingY = 2f;

    void Start () {
        for (int i=0; i<numBaskets; i++) {
            GameObject tBasketGO = Instantiate( basketPrefab ) as GameObject;
            Vector3 pos = Vector3.zero;
            pos.y = basketBottomY + ( basketSpacingY * i );
            tBasketGO.transform.position = pos;
        }
    }
}
```

- a. 本行代码在 Unity 中为检视器增加了一个函数头,用于查看在检视器中应设置的变量。后面章节的示例代码会使用 Set Dynamically 函数头完成游戏中变量的计算和设置。

本段代码实例化了 3 个纵向排列的篮筐预设的副本。

3. 在层级面板中单击主摄像机,在检视面板中将 basketPrefab 设置为之前创建的 Basket 游戏对象预设,然后单击播放按钮。你会看到这段代码在屏幕底部创建了三个篮筐。

让篮筐跟随鼠标移动

下面的代码实现了篮筐跟随鼠标移动的功能。

1. 在项目面板的篮筐预设中添加篮筐脚本。

2. 在 MonoDevelop 中打开 Basket C#脚本,输入以下代码:

```
using System.Collections;
```




```

using System.Collections.Generic;
using UnityEngine;

public class Basket : MonoBehaviour {

    void Update () {
        // 从 Input 中获取鼠标在屏幕中的当前位置
        Vector3 mousePos2D = Input.mousePosition; // a

        // 摄像机的 z 坐标决定在三维空间中鼠标沿 z 轴向前移动多远
        mousePos2D.z = -Camera.main.transform.position.z; // b

        // 将该点从二维屏幕空间转换为三维游戏世界空间
        Vector3 mousePos3D = Camera.main.ScreenToWorldPoint( mousePos2D ); // c

        // 将篮筐的 x 位置移动到鼠标处的 x 位置处
        Vector3 pos = this.transform.position;
        pos.x = mousePos3D.x;
        this.transform.position = pos;
    }
}

```

- a. 将 Input.mousePosition 赋值给 mousePos2D。这个值为屏幕坐标，即鼠标指针到屏幕左上角的像素数。Input.mousePosition 的 z 坐标始终默认为 0，因为它本质上是一个二维测量结果。
- b. 本行代码将 mousePos2D 的 z 坐标设置为主摄像机 z 坐标的相反数。在游戏中，主摄像机的 z 坐标为 -10，所以 mousePos2D.z 为 10。这会让后面出现的 ScreenToWorldPoint() 函数知道让 mousePos3D 在三维空间中向前移动多远。
- c. ScreenToWorldPoint() 将 mousePos2D 转换为屏幕三维空间中的一个位置。如果 mousePos2D.z 为 0，结果得到的 mousePos3D 的 z 坐标将在 z=-10 处（与主摄像机 z 坐标相等）。将 mousePos2D.z 设置为 10 后，mousePos3D 会被影射到三维空间中 z 坐标距离主摄像机 10 米处，使 mousePos3D.z 为 0。这对《拾苹果》游戏影响不大，但对后面的游戏很重要。如果对这个问题有疑惑，建议在 Unity 脚本参考中查看 Camera.ScreenToWorldPoint()。⁶

现在单击“播放”按钮，篮筐就可以移动了，你可以使用篮筐碰到苹果，但还不能真正接住苹果。

接住苹果

下面实现了接住苹果功能。

6. 相关内容可以参考 Unity 官网上的 ScriptReference 文档。请确认单击“C#”按钮查看 C# 文档（否则是 JavaScript 版本）。



1. 在 Basket 脚本中添加以下代码:

```
public class Basket : MonoBehaviour {
    void Update () { ... }

    void OnCollisionEnter( Collision coll ) { // a
        // 检查与篮筐碰撞的是什么对象
        GameObject collidedWith = coll.gameObject; // b
        if ( collidedWith.tag == "Apple" ) { // c
            Destroy( collidedWith );
        }
    }
}
```

- a. 只要有其他游戏对象碰撞到篮筐, 就会调用 `OnCollisionEnter` 方法, 并传递进来一个 `Collision` 参数, 其中包含了全部碰撞信息, 包括对碰撞到 Basket 碰撞器的游戏对象的一个引用。
- b. 本行代码将碰撞到的游戏对象赋值给临时变量 `collidedWith`。
- c. 检查 `collidedWith` 是否带有 "Apple" 标签, 从而确定其是否为 Apple 对象。如果 `collidedWith` 是一个 Apple 对象, 就将其销毁。现在当苹果碰到篮筐后就会消失。

2. 保存篮筐脚本, 返回 Unity 并单击“播放”按钮。

到目前为止, 游戏运行已经非常像我们要模仿的《炸弹人》游戏了, 但它还缺少图形用户界面 (graphical user interface, 简称 GUI) 元素, 例如得分或剩余生命数量。但即使没有这些元素, 《拾苹果》以目前状态也算是一个成功的原型游戏了。因为游戏原型可以让你进行调整以便达到合适的难度。

3. 请保存场景。

4. 为当前场景保存一个副本, 用于测试游戏平衡性调整。

- a. 在项目面板中单击 `_Scene_0`。
- b. 按下 `Command+D` 组合键 (在 Windows 系统中为 `Ctrl+D` 组合键), 或执行 `Edit (编辑) > Duplicate (复制)` 命令复制场景, 这会生成一个名为 `_Scene_1` 的新场景。
- c. 双击打开 `_Scene_1`。

由于它是 `_Scene_0` 的副本, 所以游戏在这个场景中没有发生变化。

在层级面板中单击苹果树, 平衡化 `_Scene_1` 变量, 同时不修改 `_Scene_0` 对应值 (对项目面板中游戏对象的任何修改都同时适用于 2 个场景), 可以提高游戏难度。当在 `_Scene_1` 中调整到合适的难度时, 请保存场景并重新打开 `_Scene_0`。如果想确认当前打开的场景, 可以查看 Unity 窗口顶部或层级面板顶部的标题栏, 其中总会有当前场景的名称。



28.4 图形用户界面（GUI）和游戏管理

游戏制作的最后一项工作是实现 GUI 和游戏管理，使其更像是一个真正的游戏。我们要添加的 GUI 元素是一个计分器，要添加的游戏管理元素则是等级和生命值。

计分器

计分器可以让玩家知道自己在游戏中获得的成就级别。

1. 在项目面板中双击，打开 `_Scene_0`。
2. 在菜单栏中执行 `GameObject>UI>Text` 命令。⁷

这是在本场景中添加的第一个 uGUI（Unity 图形用户接口）元素，用于向层级面板增加内容。首先你会看到画布（Canvas）。画布是 GUI 布局用的二维板，当查看场景面板时会发现，一个很大的 2D 框从边缘向 X 和 Y 方向延伸很远。

3. 在层级中双击画布并放大查看。画布会调整大小，匹配游戏面板尺寸，比如设置的游戏面板长宽比为 16:9，那么画布的比例也相同。你也可以单击场景面板上的“2D”按钮转换为二维视图，方便配合画布的使用。

在层级面板顶部需要添加的另一个游戏对象为 `EventSystem`。用于运转那些在 uGUI 中创建的按钮、滚动条和其他 GUI 交互元素，不过在本模型中用不到。

这里有一个名为 `Text` 的游戏对象，是 `Canvas` 的子对象，如果看不到，单击层级面板中 `Canvas` 旁的三角形展开标志显示该子对象。双击层级面板中的 `Text` 游戏对象，使之放大，`Text` 的默认颜色为黑色，因此很难在场景面板的背景中看出它的存在。

4. 在层级中选择 `Text` 游戏对象，并使用检视器面板将其重命名为 `HighScore`。
5. 按照下面步骤设置 `HighScore` 检视器，如图 28-10 所示。

a. 在 `HighScore` 检视器的 `RectTransform` 组件中：

- 设置 `Anchors` 为 `Min X=0, Min Y=1, Max X=0, Max Y=1`；
- 设置 `Pivot` 为 `X=0, Y=1`；
- 设置 `Pos` 为 `X=10, Pos Y=-6, Pos Z=0`；
- 设置 `Width=256, Height=32`。

设置完成后，再次在层级面板中双击 `HighScore`，重定位到场景面板的中心视图。

b. 在 `HighScore` 检视器的 `Text(Script)` 组件中：

- 设置 `Text` 为 `High Score: 1000`。
- 设置 `Font Style` 为 `Bold`。

7. 笔者在执行本节示例的 `GameObject>UI>Text` 命令时发现，Unity 菜单有时会出现变灰不可用的情况。如果你遇到相同问题，可以右击层级面板空白处，在弹出的快捷菜单中执行 `UI>Text` 命令。



- 设置 Font Size 为 28。
- 设置 Color 为白色，使之在游戏面板中可见。

6. 在层级面板中右击 HighScore 并在打开的快捷菜单中选择“Duplicate”选项。⁸

7. 选择新的 HighScore (1) 游戏对象，并重命名为 ScoreCounter。

8. 按照图 28-10 所示，在检视器中修改 ScoreCounter 的 RectTransform 和 Text 值。记住在 RectTransform 组件中设置 Anchors 和 Pivot 值，以及设置 Text 组件中的 Alignment 值。可以看到，在 RectTransform 中修改 Anchors 或 Pivot 时，Unity 会自动修正 Pos X，使得 ScoreCounter 在 Canvas 中的位置保持不变。要禁用 Unity 的此项功能，可以单击 ScoreCounter 检视器的 RectTransform 选区中的“R”按钮。

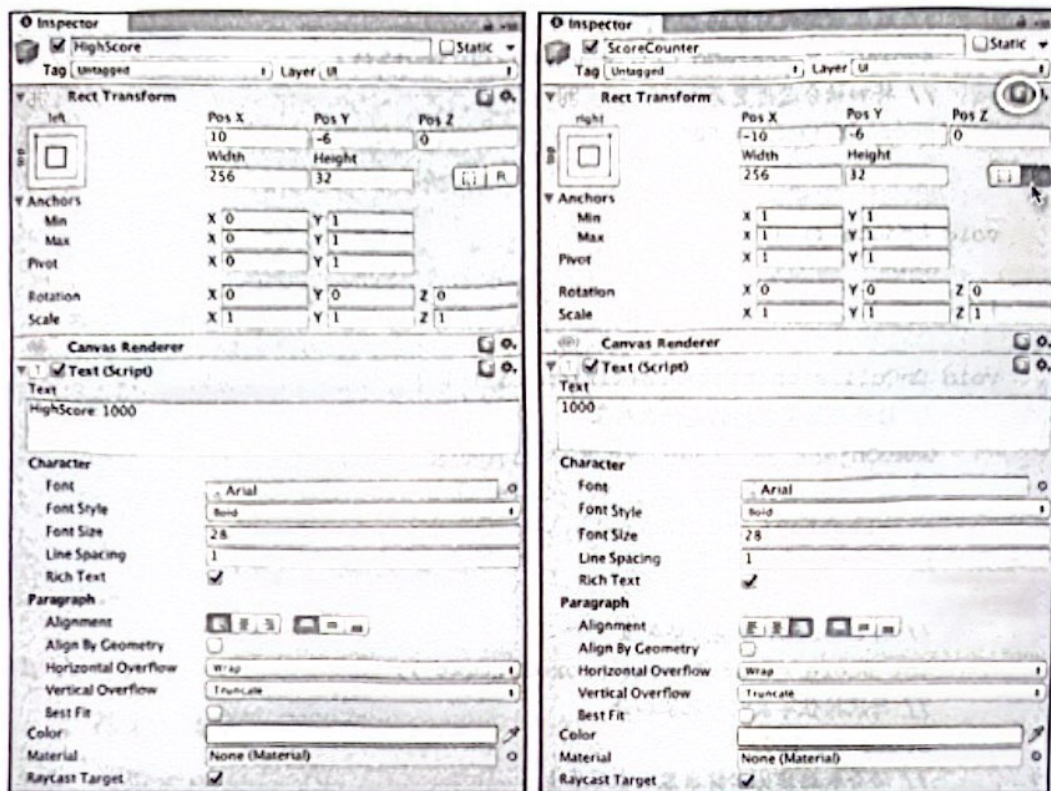


图 28-10 ScoreCounter 和 HighScore 的 RectTransform 组件及 Text 组件设置

每次接住苹果时为玩家加分

当苹果碰撞到篮筐时，Apple 和 Basket 脚本都会收到消息。在本游戏中，Basket 脚本中已经有了一个 OnCollisionEnter() 方法，所以我们会修改这部分代码，让玩家每接到一个苹果就获得一定分数。每接到一个苹果得 100 分感觉是个合适的分数（尽管笔者总认为在分数后面多加几个零有点可笑）。

1. 在 MonoDevelop 中打开 Basket 脚本，加入下面粗体字代码：

```
using System.Collections;
```

8. 你可能需要再次单击层级面板查看 HighScore (1) 的副本。




```

using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;           //本行代码为 uGUI 特征库函数           // a

public class Basket : MonoBehaviour {

    [Header("Set Dynamically")]
    public Text scoreGT;           // a

    void Start() {
        // 查找 ScoreCounter 游戏对象
        GameObject scoreGO = GameObject.Find("ScoreCounter");           // b
        // 获取该游戏对象的 GUIText 组件
        scoreGT = scoreGO.GetComponent<GUIText>();           // c
        // 将初始分数设置为 0
        scoreGT.text = "0";
    }

    void Update () {
        .....
    }

    void OnCollisionEnter( Collision coll ) {
        // 检查与篮球碰撞的是什么对象
        GameObject collidedWith = coll.gameObject;
        if ( collidedWith.tag == "Apple" ) {
            Destroy( collidedWith );
        }

        // 将 scoreGT 转换为整数值
        int score = int.Parse( scoreGT.text );           // d
        // 每次接住苹果就为玩家加分
        score += 100;
        // 将分数转换为字符串显示在屏幕上
        scoreGT.text = score.ToString();
    }
}

```

Text

- a. 请确保没有忽略这行代码，它独立于其他代码行。
- b. `GameObject.Find("ScoreCounter")` 方法在全部游戏对象中查找名为 `ScoreCounter` 的对象，并把它赋给局部变量 `scoreGO`。确认代码中的 `ScoreCounter` 没有空格或在层级中。
- c. `scoreGO.GetComponent<GUIText>()` 方法用来查找 `scoreGO` 游戏对象的 `GUIText` 组件，并赋给全局字段 `scoreGT`。在下一行中，通过这个字段将初始分数设置为 0。如果没有声明 `using UnityEngine.UI;` 代码行，



Unity 不支持定义 C# 的 Text 组件。随着 Unity 编程技术不断改进,逐步可以通过手动添加代码库以支持新特性。

- d. `int.Parse(scoreGT.text)` 获得 `ScoreCounter` 中的文字内容,并转换为整数赋值给整型变量 `score`,让 `score` 的数值增加 100 后,使用 `score.ToString()` 转换为字符串设置为 `scoreGT` 的文本内容。

未接住苹果时通知 Apple Picker 脚本

在未接住苹果时结束本回合并销毁篮筐,可以让《拾苹果》感觉更像一个真正的游戏。这时,由 Apple 负责销毁自身,这没有问题,但是 Apple 需要以某种方式将该事件通知 ApplePicker 脚本,以便让 Apple Picker 可以结束本回合并销毁其余的苹果,这涉及脚本间的相互调用。

1. 首先,请在 MonoDevelop 中对 Apple 脚本中做如下修改:

```
public class Apple : MonoBehaviour {
    [Header("Set in Inspector")]
    public static float bottomY = -20f;

    void Update () {
        if ( transform.position.y < bottomY ) { //a
            Destroy( this.gameObject );

            // 获取对主摄像机的 ApplePicker 组件的引用
            ApplePicker apScript = Camera.main.GetComponent<ApplePicker>(); // b
            // 调用 apScript 的 AppleDestroyed 方法
            apScript.AppleDestroyed(); // c
        }
    }
}
```

- a. 注意后面的代码都是在 if 语句中的。
- b. 获取对主摄像机的 ApplePicker (Script) 组件的引用。因为 Camera 类有一个内置的静态变量 `Camera.main`,用来引用主摄像机,所以不必使用 `GameObject.Find("Main Camera")` 语句获取对主摄像机的引用。`GetComponent<applePicker>()` 用于获取对主摄像机的 ApplePicker (Script) 组件的引用,并将其赋给 `apScript`。然后,就可以访问绑定到主摄像机上的 ApplePicker 实例中的全局变量和方法了。
- c. 调用 ApplePicker 类的 `AppleDestroyed()` 方法,目前这个方法还不存在,因此在 MonoDevelop 中显示为空色行,除非定义 `AppleDestroyed()` 才可以在 Unity 中运行游戏。

2. 目前 ApplePicker 脚本中还不存在全局方法 `AppleDestroyed()`,需要在 MonoDevelop 中打开 ApplePicker 脚本,在其中加入以下粗体代码:




```

public class ApplePicker : MonoBehaviour {
    ...

    void Start () { ... }

    public void AppleDestroyed() { // a
        // 销毁所有下落中的苹果
        GameObject[] tAppleArray=GameObject.FindGameObjectsWithTag("Apple"); // b
        foreach ( GameObject tGO in tAppleArray ) {
            Destroy( tGO );
        }
    }
}

```

- a. AppleDestroyed() 方法必须声明为 public，这样其他类（例如 Apple）才能够调用。在默认情况下，方法为私有，不能被其他类所调用或者查看。
- b. GameObject.FindGameObjectsWithTag("Apple") 返回一个数组，其中包含了当前已经存在的所有 Apple 对象。⁹后面的 foreach 循环语句遍历每个 Apple 游戏对象并将其销毁。

保存 MonoDevelop 中所有脚本。现在定义了 AppleDestroyed() 脚本，在 Unity 中可以继续玩游戏了。

当未接住苹果时，销毁一个篮筐

本场景的最后一部分代码负责在未接住苹果时销毁一个篮筐，并在所有篮筐都被销毁时，停止运行游戏。请在 ApplePicker 脚本中做以下修改（仅在本示例中列出了所有代码）：

```

using System.Collections;
using System.Collections.Generic; // a
using UnityEngine;
using UnityEngine.SceneManagement; // b

public class ApplePicker : MonoBehaviour {

    [Header("Set in Inspector")]
    public GameObject basketPrefab;
    public int numBaskets = 3;
    public float basketBottomY = -14f;
    public float basketSpacingY = 2f;
}

```

9. GameObject.FindGameObjectsWithTag() 一般会消耗很多处理器资源，所以不推荐在 Update() 或 FixedUpdate() 函数中使用它。但本例只会在游戏玩家丢失篮筐（并且游戏被设计为在此处变慢）时出现这种情况，因此可以使用这个函数。




```

public List<GameObject> basketList;

void Start () {
    basketList = new List<GameObject>(); //c
    for (int i=0; i<numBaskets; i++) {
        GameObject tBasketGO =Instantiate<GameObject>( basketPrefab );
        Vector3 pos = Vector3.zero;
        pos.y = basketBottomY + ( basketSpacingY * i );
        tBasketGO.transform.position = pos;
        basketList.Add( tBasketGO ); // d
    }
}

public void AppleDestroyed() {
    // 销毁所有下落中的苹果
    GameObject[] tAppleArray =
GameObject.FindGameObjectsWithTag( "Apple" );
    foreach ( GameObject tGO in tAppleArray ) {
        Destroy( tGO );
    }
    // 销毁一个篮筐
    // 获取basketList 中最后一个篮筐的序号
    int basketIndex = basketList.Count-1;
    // 取得对该篮筐的引用
    GameObject tBasketGO = basketList[basketIndex];
    // 从列表中销毁该篮筐并销毁该游戏对象
    basketList.RemoveAt( basketIndex );
    Destroy( tBasketGO );
}

```

- a. 这里会把所有 Basket 游戏对象存储在一个 List 中，所以需要引用 System.Collections.Generic 代码库（请参考第 23 章中关于 List 的说明）。
public List<GameObject> basketList 在类的头部进行声明，在 Start() 函数中定义并初始化。
- b. 本行会在下一节中使用到。
- c. 定义 basketList 作为新的 List<GameObject>。尽管//b 行代码已经声明，而声明后的 basketList 值为 null。本行的初始化操作使之成为可用的列表。
- d. 在 for 循环的尾部新增加了一行代码，将 basket 添加到 basketList 中。basket 添加到 basketList 中的顺序与其创建顺序相同，即从下往上依次添加。
- e. 在 AppleDestroyed() 方法中新增加了一段代码，用于销毁一个篮筐。因为篮筐的添加顺序为从下往上，所以要保证先销毁 List 中的最后一个篮筐（篮筐的销毁顺序为从上向下）。



此时玩游戏并用完筐筐，Unity 会抛出 `ndexOutOfRangeException` 异常消息。

添加最高得分纪录

本节使用前文创建的 `HighScore` 文本游戏对象。

1. 新建一个名为 `HighScore` 的 C# 脚本，将其绑定到层级面板中的 `HighScore` 游戏对象上。

2. 在 `MonoDevelop` 中打开 `HighScore` 脚本并添加以下代码：

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;           //记住添加 uGUI 函数库

public class HighScore : MonoBehaviour {
    static public int score = 1000;    //a

    void Update () {                //b
        Text gt = this.GetComponent<Text>();
        gt.text = "High Score: "+score;
    }
}
```

- 把整型变量 `score` 声明为全局静态变量，就可以在任何脚本中使用 `HighScore.score` 访问它。这是静态变量的优势，我们会在本书其他游戏原型中经常使用。
- `Update()` 中的代码只是用于显示 `Text` 组件中的得分。这里不需要调用 `score` 的 `ToString()` 方法，因为使用 `+` 号把一个字符串与另一种数据类型的变量（这里是连接 `"High Score: "` 字符串和整型变量 `score`）相连接时，会隐式调用（自动调用）`ToString()` 方法。

3. 请打开 `Basket` 脚本，添加以下代码，学习全局静态变量的用法：

```
public class Basket : MonoBehaviour {
    ...
    void OnCollisionEnter( Collision coll ) {
        ...
        if ( collidedWith.tag == "Apple" ) {
            ...
            // 把 score 转换为字符串并显示
            scoreGT.text = score.ToString();

            // 监视最高分
            if (score > HighScore.score) {
                HighScore.score = score;
            }
        }
    }
}
```



现在，每当目前得分超出 `HighScore.score` 时，就会重新设置 `HighScore.score`。

4. 打开 `ApplePicker` 脚本，加入下面代码，在玩家失去所有篮筐后结束游戏。避免抛出前文提到的 `IndexOutOfRangeException` 异常消息。

```
public class ApplePicker : MonoBehaviour {
    ...
    public void AppleDestroyed() {
        ...
        // 从列表销毁篮筐并销毁游戏对象

        basketList.RemoveAt( basketIndex );
        Destroy( tBasketGO );

        // 如果没有篮筐剩余，重新开始游戏
        if ( basketList.Count == 0 ) {
            SceneManager.LoadScene ( "_Scene_0" );    //a
        }
    }
}
```

- a. `SceneManager.LoadScene ( "_Scene_0" )` 会重新加载场景 `_Scene_0`。前提是当苹果先于篮筐掉落时，已经在销毁篮筐之前添加了 `using UnityEngine.SceneManagement;` 代码。重新加载场景使得游戏回到初始状态。¹⁰

5. 现在已经修改了一些脚本代码。你是否记得保存了？如果没有或者不确定，可以像笔者一样，在 `MonoDevelop` 菜单栏中执行 `File > Save All` 命令，保存所有已修改但未保存的脚本。如果“`Save All`”选项为灰色不可用状态，证明已经保存了所有修改。

在 `PlayerPrefs` 中保存最高得分

因为 `HighScore.score` 是一个静态变量，所以不会在重新开始游戏时被重置。也就是说，在进入游戏下一回合时，最高得分纪录不会改变。然而，无论何时停止游戏（再次单击“播放”按钮），`HighScore.score` 会恢复为初始值。要解决这一问题，需要用到 Unity 的 `PlayerPrefs` 功能。`PlayerPrefs` 可以将 Unity 脚本中的信息保存到计算机上，以供将来调用，并且即使游戏结束后也不会被销毁。`PlayerPrefs` 也可以在 Unity 编辑器、编译器和 WebGL 中使用，因此获得的最高得分可以传递给运转在同一台计算机上的其他对象。

10. 在 Unity 2017 及之前多个版本中，重新加载层级时常常会出现一个错误。如果使用 `SceneManager` 重载时，场景变得更暗，那么证明你遇到了相同问题。在 Unity 修复这个 Bug 之前，临时的解决方案是禁用自动光（预先计算场景大致光线）。在 Unity 菜单栏中执行 `Window > Lighting > Settings` 命令，打开照明面板可完成，取消选中照明面板底部的“`Auto Generate`”选项（在“`Generate Lighting`”按钮旁），然后单击一次“`Generate Lighting`”按钮，手动重建灯光，等待完成，并关闭灯光窗口。这个 Bug 只出现在 Unity 编辑器中，不影响任何 WebGL 或独立构建的应用程序。



1. 打开 HighScore 脚本并加入下面以粗体字代码：

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;           //记住添加 uGUI 函数库

public class HighScore : MonoBehaviour {
    static public int score = 1000;

    void Awake() {               // a
        // 如果 PlayerPrefs HighScore 已经存在，则读取其值
        if (PlayerPrefs.HasKey("HighScore")) {           // b
            score = PlayerPrefs.GetInt("HighScore");
        }
        // 将最高得分赋给 HighScore
        PlayerPrefs.SetInt("HighScore", score);           // c
    }

    void Update () {
        Text gt = this.GetComponent<Text>();
        gt.text = "High Score: "+score;
        // 如有必要，则更新 PlayerPrefs HighScore
        if (score > PlayerPrefs.GetInt("HighScore")) {    // d
            PlayerPrefs.SetInt("HighScore", score);
        }
    }
}

```

- a. Awake() 是 Unity MonoBehaviour 的内置方法（类似于 Start() 或 Update()），在首次创建 HighScore 实例时运行（因此 Awake() 总在 Start() 之前发生）。
- b. PlayerPrefs 是一个关键字和数值的字典，可以通过关键字（即独一无二的字符串）引用值。在本例中引用的关键字为 HighScore。本行检查 PlayerPrefs 中是否已经存在 HighScore，如果存在，就读取它的值。
- c. Awake() 中的最后一行将 score 的当前值赋给 PlayerPrefs 中的 HighScore 关键字。如果 HighScore 已经存在，该语句会将数值写入 PlayerPrefs。如果 HighScore 关键字不存在，该语句会创建 HighScore 关键字。
- d. 添加上述代码之后，Update() 每帧都会检查当前的 HighScore.score 是否高于 PlayerPrefs 中存储的最高得分，如果确实如此，则更新 PlayerPrefs。

使用 PlayerPrefs 可以在本计算机上保存《拾苹果》的最高得分，即使结束游戏，退出 Unity，甚至重启计算机后，最高得分也能保存。

2. 在 MonoDevelop 中再次执行 Save All 命令，确保保存所有修改。回到 Unity 并单击“播放”按钮。



至此,可以完整运行游戏并显示得分和最高分。如果想刷新最高分,停止游戏并重启,可以看到显示为最新的最高分。

28.5 本章小结

现在,你拥有了一个类似于 Activision 公司的《炸弹人》游戏的原型。尽管本游戏仍然缺少一些元素(例如持续增加游戏难度、游戏界面宽度变化等),但有了足够的编程经验后,你可以在游戏中添加这些元素。

后续工作

将来可以在游戏原型中添加一些元素,以下是相关说明。学习编程最好的方法就是先模仿例如本章的示例代码,然后尝试添加自己的修改。

- **欢迎界面:**可以在单独的场景中创建一个欢迎界面,并添加一个启动画面和一个“开始”按钮。由“开始”按钮调用 `SceneManager.LoadScene("_Scene_0")` 开始游戏。记住使用 `SceneManager` 之前,一定要在脚本头部添加 `using UnityEngine.SceneManagement` 语句。
- **Game Over 界面:**可以增加一个 Game Over 界面。可以在 Game Over 界面中展示玩家的最终得分,并让玩家知道自己是否打破了原来的最高得分纪录。Game Over 界面中还应该有一个“重新开始”按钮,用它调用 `SceneManager.LoadScene("_Scene_0")` 重新开始游戏。
- **变化的难度:**后面几个游戏原型中会涉及难度等级的变化,如果希望增加这个游戏的难度,可以把不同难度等级对应的 `AppleTree` 的各个字段值(例如 `speed`、`chanceToChangeDirections`、`secondsBetweenAppleDrops` 等)保存到一个数组或 `List` 中。使 `List` 中的每个元素对应不同难度等级下的各变量值,让第 0 个元素对应难度最小的级别,最后一个元素对应难度最大的级别。用变量 `level` 代表难度等级,在玩家玩游戏的过程中,每隔一段时间就让 `level` 增加 1,同时使用这个变量作为 `List` 的索引值,这样当 `level=0` 时,则使用 `List` 中第 0 个元素的各个变量。

如果想在游戏中添加一个开始场景或结束场景,需要将游戏中的每一个场景添加到 `Build Settings` 场景列表。在 `Unity` 中打开每个场景,然后在 `Unity` 菜单中执行 `File > Build Settings` 命令。在打开的 `Build Settings` 窗口中单击“Add Open Scenes”按钮,当前场景的名称将被添加到场景的 `Build` 列表中。如果确实对此游戏创建了一个构建,则游戏在开始运行时首先加载编号为 0 的场景。

